

1. Algoritmo Greedy para encontrar minimo tiempo

En este trabajo realizaremos el analisis teorico y empirico del algoritmo Greedy propuesto para resolver el problema de scheduling de videos.

1.1. Algoritmo Greedy

A continuacion se muestra el codigo de solucion Greedy del problema.

```

1 def tiemposminimos(n_equipos, scaloni, ayudantes):
2     equipos=[]
3     for i in range(1,n_equipos+1):
4         equipos.append(i)
5     aux=list(zip(scaloni,ayudantes,equipos))
6     aux=sorted(aux,key=lambda x:x[1],reverse=True)
7     tiempo_total=0
8     tiempo_ite=0
9     orden=[]
10    for (scaloni,ayudantes,equipo) in aux:
11        tiempo_ite+=scaloni
12        tiempo_total=max(tiempo_total,tiempo_ite+ayudantes)
13        orden.append(equipo)
14    return tiempo_total,orden

```

1.2. Regla Greedy

En cada iteracion de los videos que me restan por analizar elijo el video cuyo tiempo de ayudante sea mayor.

1.3. Optimo Local

Para cada video i , se calcula su tiempo de analisis como:

tiempo acumulado de scaloni (hasta ahora) + $s_i + a_i$

Si esta suma representa un nuevo tiempo maximo de analisis, con respecto a un tiempo total previo, entonces el tiempo total se actualiza por esta suma.

1.4. Solución Optima

Demostración por inversiones:

Tenemos 2 elementos en la solución óptima tales que $a_i < a_j$ y $i < j$, es decir, que un elemento anterior tiene un tiempo de ayudante menor al posterior. Esto va en contra de nuestro algoritmo, por lo que se necesitaría una inversión para llegar a la solución optima propuesta por nuestro algoritmo.

El orden, al no tener inversiones, implica que solo varían posiciones de videos consecutivos cuyo tiempo de análisis de ayudante sea igual. Y tiempo total maximo dado por el ultimo elemento consecutivo.

Supongo que un orden con inversiones es optimo, para que esto sea mas que una suposición tengo que descartar un problema, si al cambiar el orden de videos consecutivos el tiempo total de la solución aumenta:

T = tiempo acumulado (hasta ahora)

s =scaloni

a =ayudantes

Pre inversion: $i - > j$

$$tiempo_j = T + si + sj$$

$$Totaltiempo1 = \max(T + si + ai, T + si + sj + aj)$$

Post inversion: $j - > i$

$$tiempo_i = T + sj + si$$

$$Totaltiempo2 = \max(T + sj + aj, T + sj + si + ai)$$

Teniendo en cuenta $ai < aj$ se cumple:

$$T + si + sj + aj > T + sj + aj$$

$$T + si + sj + aj > T + si + sj + ai$$

Por lo que:

$$Totaltiempo1 > Totaltiempo2$$

El tiempo máximo total de i no puede ser mayor que el tiempo máximo de total j. Demostramos que la solución no empeora, es optima ya que no puede ser peor que el tiempo total del problema.

1.5. Complejidad Algoritmo

1.6. Análisis tiempos

Proponemos cuatro casos:

1.6.1. $ai \gg si$

En este caso, se puede observar mediante el ejemplo de ejecucion ayudantes mayor.txt que el tiempo total obtenido se basa mayormente en los valores de los ayudantes, el máximo suele alcanzarse en las primeras iteraciones si no en la primera.

Ejemplo:

$$si = [40, 30, 20, 60]$$

$$ai = [100, 200, 500, 350]$$

Valores en cada Iteracion:

$$\text{tiempo total:} 520, \text{ tiempo scaloni:} 20, \text{ comparacion : } 0 < 520$$

$$\text{tiempo total:} 520, \text{ tiempo scaloni:} 80, \text{ comparacion : } 520 > 350 + 80$$

$$\text{tiempo total:} 520, \text{ tiempo scaloni:} 110, \text{ comparacion : } 520 > 110 + 200$$

$$\text{tiempo total:} 520, \text{ tiempo scaloni:} 150, \text{ comparacion : } 520 > 150 + 100$$

Como se puede observar solo basta la primera iteracion para obtener el tiempo total necesario para analizar todos los videos.

1.6.2. $ai \approx si$

Si la variacion de tiempo es pequeña, es decir que los tiempos son similares, el tiempo total va a depender tanto de los valores acumulados de scaloni como los valores de los ayudantes. Tenemos el ejemplo de ejecucion tiempos similares.txt donde se corrobora el analisis del tiempo optimo.

Ejemplo:

$$si = [21, 38, 92, 100, 55, 27, 50, 41]$$

$$ai = [17, 42, 96, 96, 55, 24, 43, 47]$$

Valores en cada iteracion:

$$\text{tiempo total:} 188, \text{ tiempo scaloni:} 92, \text{ comparacion : } 0 < 96 + 92$$

tiempo total:288, tiempo scaloni:192, *comparacion* : $188 < 96 + 192$
 tiempo total:306, tiempo scaloni:251, *comparacion* : $288 < 55 + 251$
 tiempo total:339, tiempo scaloni:292, *comparacion* : $306 < 41 + 292$
 tiempo total:385, tiempo scaloni:342, *comparacion* : $339 < 43 + 342$
 tiempo total:422, tiempo scaloni:380, *comparacion* : $385 < 42 + 380$
 tiempo total:431, tiempo scaloni:407, *comparacion* : $422 < 24 + 407$
 tiempo total:445, tiempo scaloni:428, *comparacion* : $431 < 17 + 428$

Como los valores de analisis de los ayudantes varian en un rango pequeño con respecto a los valores de scaloni, estos ultimos van a obligar al algoritmo a actualizar el tiempo total en cada iteracion ya al ser acumulativos la suma con su par a_i va a ser siempre mayor que el anterior.

1.6.3. $si \gg ai$

El tiempo total va a ser dominado por la suma total de tiempos de scaloni, el tiempo de ayudantes es descartable, solo se tiene en cuenta el tiempo del ultimo video. Es igual a la variacion de tiempo pequeña anteriormente explicada. El ejemplo de ejecucion scaloni mayor.txt nos da una solucion en linea con el analisis.

Ejemplo:

$si=[1000,400,300,1500,600,700,100,630,22900]$

$ai=[100,120,78,250,150,200,20,47,2900]$

Valores en cada iteracion:

tiempo total:25800, tiempo scaloni:22900, *comparacion* : $0 < 2900 + 22900$
 tiempo total:25800, tiempo scaloni:24400, *comparacion* : $25800 > 250 + 24400$
 tiempo total:25800, tiempo scaloni:25100, *comparacion* : $25800 > 200 + 25100$
 tiempo total:25850, tiempo scaloni:25700, *comparacion* : $25800 < 150 + 25700$
 tiempo total:26220, tiempo scaloni:26100, *comparacion* : $25850 < 120 + 26100$
 tiempo total:27200, tiempo scaloni:27100, *comparacion* : $26220 < 100 + 27100$
 tiempo total:27478, tiempo scaloni:27400, *comparacion* : $27200 < 78 + 27400$
 tiempo total:28077, tiempo scaloni:28030, *comparacion* : $27478 < 47 + 28030$
 tiempo total:28150, tiempo scaloni:28130, *comparacion* : $28077 < 20 + 28130$
 tiempo total:28150

Como se observa en el ejemplo el tiempo total se obtiene en la ultima iteracion, luego de acumular todos los valores de scaloni y sumarle el tiempo de analisis de ayudantes del ultimo video se obtiene el maximo. Ya que ese tiempo total depende fundamentalmente de los extremadamente mayores valores de scaloni con respecto a los valores de ayudantes.

1.6.4. $si = ai$

Llegamos a una conclusion similar al segundo caso, el tiempo total va a depender de igual manera de los valores de scaloni como de los ayudantes. El maximo no se puede obtener de una iteracion especifica. Ejemplo de ejecucion tiempos iguales.txt nos verifica esta solucion.

Ejemplo:

$si=[1000,1000,700,100,20,5]$

$ai=[1000,1000,700,100,20,5]$

Valores en cada iteracion:

tiempo total:2000, tiempo scaloni:1000, *comparacion* : $0 < 1000 + 1000$

tiempo total:3000, tiempo scaloni:2000, *comparacion* : $2000 < 1000 + 2000$

tiempo total:3400, tiempo scaloni:2700, *comparacion* : $3000 < 700 + 2700$

tiempo total:3400, tiempo scaloni:2800, *comparacion* : $3400 > 100 + 2800$

tiempo total:3400, tiempo scaloni:2820, *comparacion* : $3400 > 20 + 2820$

tiempo total:3400, tiempo scaloni:2825, *comparacion* : $3400 > 5 + 2825$

tiempo total:3400

En este ejemplo de ejecucion el tiempo total se obtiene en la iteracion 3, no depende exactamente de la relacion entre s_i y a_i sino de la variacion de valores en el arreglo en general.